

AY05: Repaso Prueba I1

Equipo docente:)<3



Entorno y Herramientas

Dominando los Paths, la Terminal y el Control de Versiones.

Paths (Rutas)

Un **Path** es la dirección exacta donde vive un archivo o carpeta en tu computador.

Rutas Absolutas: Empiezan desde la raíz principal del disco duro (ej. /Usuarios/Tú/Docs). ¡Son precisas y nunca fallan!

Rutas Relativas: Se basan en dónde estás ubicado **ahora mismo** en la terminal. Si ya estás en la carpeta Docs, la ruta relativa es simplemente el nombre del archivo (ej. tarea.py).



Comandos de Terminal Esenciales

Comando	¿Qué hace?
pwd	Te muestra la ruta absoluta de la carpeta en la que estás actualmente.
ls o dir	Lista todos los archivos y carpetas que hay donde estás parado.
cd [carpeta]	Cambia de directorio (entras a la carpeta que le indiques). cd .. te saca de la carpeta.
mkdir [nombre]	Crea una carpeta nueva completamente vacía.
rm [archivo]	Borra el archivo indicado (¡Cuidado, no pasa por la papelera!).
cp [ori] [des]	Copia un archivo desde el origen (ori) hasta la nueva ruta destino (des).
mv [ori] [des]	Mueve un archivo. También se usa para renombrar archivos.
clear	Limpia todo el texto de la pantalla de la terminal.

Flujo Lineal de Git

💡 **Tip de oro:** Usa "git status" constantemente para ver qué archivos cambiaste y en qué paso vas.

1. clone / pull

Trae o actualiza el código desde la nube (GitHub) a tu PC local.



2. add

Selecciona y "prepara" los archivos modificados que vas a querer guardar.



3. commit

Crea un "punto de guardado" oficial con un mensaje corto explicando tus cambios.



4. push

Sube todos tus nuevos puntos de guardado a la nube para que no se pierdan.



Estructuras de Datos

Organización inteligente para manipular la información.

Tuplas y NamedTuples

- **Inmutables:** Una vez creadas, no puedes cambiar, añadir ni borrar sus elementos. Son cajas fuertes.
- **Secuenciales:** Mantienen un orden estricto (puedes usar índices como [0]).
- **Heterogéneas:** Pueden guardar distintos tipos de datos (números, textos) juntos.
- **NamedTuples:** Funcionan igual, pero te permiten acceder a los datos usando **nombres** en vez de números, haciendo el código súper fácil de leer (ej. mascota.edad).



El Orden de Atención

Stacks (Pilas)

LIFO (Último en entrar, primero en salir):

Imagina una pila de platos. Siempre lavas el último plato que pusieron encima de la torre.

En Python, usamos una lista normal. Agregamos con `append()` y quitamos con `pop()`.

Queues (Colas)

FIFO (Primero en entrar, primero en salir):

Imagina la fila del banco. El primero en llegar es el primero en ser atendido.

En Python usamos `deque`. Agregamos al final con `append()` y sacamos el primero con `popleft()`.

Diccionarios, Sets y Argumentos



Diccionarios

Asocian **Llave-Valor**.
Rápidos para buscar gracias
al *hash*.

Ojo: Las llaves no se pueden
repetir y deben ser
inmutables.



Sets (Conjuntos)

Estructuras matemáticas **sin
elementos repetidos**. No
tienen orden, pero son los
más rápidos para verificar si
un dato ya existe adentro.



*Args y **Kwargs

*args junta argumentos
suelos en una Tupla.

**kwargs junta parámetros
con nombre en un
Diccionario.

OOP y Excepciones

Herencia, Flujos de Errores e
Iteradores.

Diseño Orientado a Objetos (OOP)



Herencia

Una clase "Hija" recicla atributos y métodos de un "Padre" para no repetir código (Ej. Animal → Gato). Usamos `super().__init__()`.



Polimorfismo

Clases distintas tienen un método con el mismo nombre, pero hacen cosas distintas. (Un perro ladra, un gato maúlla al llamar `hablar()`).



@property

Convierte un método para que se use como si fuera un atributo. Permite **validar** los datos en secreto antes de guardarlos.

El Flujo Lineal de Excepciones

Atrapar errores evita que el programa colapse de golpe por un mal dato del usuario.

1. try

Intentamos ejecutar el código principal, sabiendo que es "riesgoso" y podría fallar.

3. else

Solo entramos aquí si el try fue un éxito rotundo (no hubo ningún error).

2. except

Solo entramos aquí si el try falla. Capturamos el error para salvar el programa.

4. finally

Se ejecuta **siempre** al final, haya habido un error catastrófico o éxito.

¡Cuidado con errores en el Except!

Si ocurre un nuevo error **dentro** de un bloque except, los siguientes bloques except **NO lo capturarán**.

¿Por qué? Porque los bloques except solo actúan como salvavidas de lo que pasa dentro del bloque try inicial, no de los otros salvavidas.

Si un salvavidas tiene una fuga, el programa se cae irreparablemente.

Iterables e Iteradores

El Iterable

Representa la colección en sí (ej. una lista o mazo de cartas).

- Implementa obligatoriamente el método mágico `__iter__()`.
- **¿Qué hace?:** Retorna un **iterador nuevo** cada vez que se llama.
- Gracias a esto, puedes recorrer el iterable **infinitas veces**.

El Iterador

Es el "puntero" que va recordando en qué posición del recorrido vas.

- Implementa `__iter__()` (que se retorna a sí mismo) y `__next__()`.
- **¿Qué hace?:** `__next__()` saca un elemento a la vez. Al acabarse los datos, levanta `StopIteration`.
- Se **agota** (se consume una sola vez).

¡Mucho Éxito en tu I1!

Concéntrate en entender la lógica de los flujos.
La memoria sintáctica se construye practicando.